

Universidad Nacional Autónoma de México

Ingeniería en Computación

Programación Orientada a Objetos

Práctica 1

ALUMNOS:

323078401

323115296

323082392

323139441

323253916

Grupo:

7

Semestre:

2026-II

México, CDMX. 2026

Índice

1. Introducción	2
2. Marco Teórico	2
3. Desarrollo	3
3.1. Respuestas de la LLM	3
3.2. Explicación del código Calculadora	4
4. Resultados	5
5. Conclusiones	6

1. Introducción

- **Planteamiento del problema.** Preguntarle a nuestra LLM entrenada sobre posibles soluciones para desarrollar una aplicación de software en Java con interfaz gráfica parecida a un editor de dibujo básico (similar a Paint). Y crear un programa de java simulando a una calculadora funcional que realice las operaciones: suma, resta, división, multiplicación, potencia, raíz y módulo.
- **Motivación.** Analizar los conceptos utilizados y el código generado por una LLM frente a requerimientos específicos a base de la salida de un programa. Además de saber implementar operaciones aritméticas en java.
- **Objetivos.** Los objetivos de esta práctica son:
 - Analizar la resolución de un problema específico de una LLM mediante la descripción de la salida.
 - Realizar un programa que simule una calculadora.

2. Marco Teórico

En la parte relacionada con la calculadora, para la implementación de cálculos numéricos avanzados se requiere el empleo del método `pow` de la clase `Math` para procesar potencias y raíces mediante exponentes fraccionarios, haciendo conversiones explícitas (casting) de `double` a `float` para asegurar el ajuste exacto del tipo de dato sin pérdidas inesperadas [1].

Para mantener el flujo interactivo de la aplicación se utilizan estructuras de control de flujo, destacando el uso de ciclos `do-while` para la persistencia del menú principal, estructuras `switch-case` para la selección de operaciones aritméticas, e instrucciones de salto como `break` para detener y salir de un bucle, y `continue` para saltar la iteración actual [2, 3].

La captura de datos desde la terminal se lleva a cabo mediante la clase `Scanner` y sus métodos `nextInt()` y `nextFloat()`, aparte de cerrar el flujo de entrada de datos desde la terminal mediante el método `close()` para evitar fugas de memoria [4].

Por otra parte, los resultados de la interfaz gráfica descrita por la LLM se basa en la programación orientada a objetos (herencia y polimorfismo), la cual propone una estructura limpia basada en una clase base abstracta de la cual derivan clases concretas que sobrescriben el método de dibujo (`@Override`), permitiendo gestionar el lienzo de forma desacoplada y sin recurrir a bifurcaciones condicionales [5].

El manejo de estructuras de datos dinámicas y persistencia en el lienzo lo hace mediante el almacenamiento del historial de trazos en una `ArrayList`, lo que ayuda con la persistencia del estado gráfico ante eventos de redibujado y permite la función de deshacer [6].

La respuesta interactiva ante las acciones del usuario responde al manejo de eventos de ratón, capturando coordenadas mientras se ejecuta implementando `MouseListener` o adaptadores como `MouseAdapter` durante eventos de clic y arrastre [7].

Finalmente, la representación visual del lienzo gráfico recurre a componentes de la API gráfica de Java como `JFrame`, `JPanel` y `Graphics` para la gestión de ventanas y la ejecución del ciclo de pintado mediante el método `paintComponent` [8, 9].

3. Desarrollo

3.1. Respuestas de la LLM

■ Análisis de la IA 1 (Resultado 1)

Esta respuesta se centra en la interfaz gráfica y la teoría de eventos, explicando cómo funciona una ventana desde cero. Utiliza conceptos como:

- Swing y AWT: Librerías nativas de Java (`JFrame`, `JPanel`, `Graphics`) para crear ventanas y dibujar.
- Programación Orientada a Eventos: Explica el ciclo de vida de una acción (el usuario hace clic → se genera un evento → el programa responde → se redibuja el lienzo). Usa variables simples (`int` figura, `boolean` relleno) para decidir qué dibujar.

■ Análisis de la IA 2 (Resultado 2)

Esta IA realiza un puente directo entre la programación estructurada (C) y Java por los antecedentes del usuario. Los conceptos que usa son:

- Estructuras de datos dinámicas: Usa una lista para guardar el historial de figuras. Esto permite implementar la función de deshacer eliminando el último elemento de la lista.
- Adaptadores de eventos: Implementa el código necesario para capturar las coordenadas al arrastrar el ratón.

■ Análisis de la IA 3 (Resultado 3)

Esta respuesta es la más teórica y se enfoca profundamente en la arquitectura Orientada a Objetos, explicando el porqué detrás de la estructura del código. Los conceptos que usa son:

- Clases Abstractas y Herencia: Define un molde principal (`Figura abstracta`) y moldes específicos.
- Polimorfismo: En lugar de usar un switch, la lista guarda objetos genéricos tipo `Figura` y manda llamar al método `dibujar()`. Cada clase hija sabe cómo dibujarse a sí misma mediante la etiqueta `@Override`.

3.2. Explicación del código Calculadora

El programa inicia importando la clase Scanner y dentro del main se abre el objeto Scanner para la lectura de datos de entrada desde la terminal y se declaran las variables requeridas para la opción elegida por el usuario, los dos números a solicitar, el resultado y los resultados de la división y multiplicación. Posteriormente, el flujo ingresa a un bucle principal do-while donde se muestra el menú de opciones con las operaciones aritméticas: suma, resta, multiplicación, división, potencia, raíz y módulo.

Antes de solicitar los datos, la aplicación realiza una validación de la opción seleccionada por el usuario: si se presiona la opción de salida, se imprime el mensaje correspondiente y se interrumpe la ejecución con una sentencia break; si la opción es inválida o fuera del rango definido, se ejecuta un continue que reinicia el ciclo sin solicitar operandos de forma innecesaria.

Al ingresar una opción válida, se le pide al usuario ingresar los números a operar y se procesan dentro de una estructura condicional switch, ejecutando la operación seleccionada, calculando el resultado numérico e imprimiéndolo. Para la suma, se hace la operación y se imprime. En el caso de la resta pregunta qué resta se quiere realizar, el primer número menos el segundo o viceversa, y realiza la resta seleccionada imprimiéndola. Con la multiplicación solo multiplica ambos números y muestra el resultado. En la división primero verifica que el divisor no sea 0 para dividir y mostrar el resultado, de lo contrario se indica que no es posible dividir entre 0.

Para la potencia se usa el método Math.pow para calcular la potencia y mostrarla. La raíz verifica que el índice no sea 0 debido a que se usa el mismo método para la potencia pero en fracción, impidiendo que el divisor sea 0. Por último, para la operación módulo de igual manera se verifica que no sea 0 el divisor debido a que se muestra el resultado de la división entera y el residuo. Concluido el caso específico, el flujo evalúa la condición del ciclo do-while para permitir realizar nuevos cálculos o finalizar el ciclo, cerrando el flujo de entrada de datos de la terminal.

4. Resultados

```
Resultado: 5.0

1. Suma
2. Resta
3. Multiplicacion
4. Division
5. Exponente
6. Raiz
7. Modulo
8. Salir
Ingresa opcion deseada: 2
Ingresa el primer numero: 4
Ingresa el segundo numero: 5

Que resta deseas realizar?
1. 4.0 - 5.0
2. 5.0 - 4.0
Selecciona una opcion: 2
Resultado: 1.0

1. Suma
2. Resta
3. Multiplicacion
4. Division
5. Exponente
6. Raiz
7. Modulo
8. Salir
Ingresa opcion deseada: 
```

Figura 1: Muestra el menú con los resultados de las operaciones del programa y te regresa el resultado y el menú

Ahora necesito que me des una idea de como hacer un programa tipo "paint" en el cual se tiene un lienzo en blanco y tiene opciones de escribir, poner figuras geometricas básicas, poner colores, poder borrar y un botón para

Para construir un Paint en Java desde tu base en C, la gran diferencia conceptual es que ya no piensas en una secuencia de funciones estructuradas (`main()`, `while(1)`, `scanf`), sino en **Objetos** que tienen atributos y métodos, y en un sistema que reacciona a eventos del usuario (como dar clic o arrastrar el ratón).

En Java utilizaremos la biblioteca estándar de interfaces gráficas llamada **Swing** (y su motor de dibujo **AWT**).

1. El cambio de mentalidad: De C a Java (POO)

En C, si querías representar un círculo, hacías una estructura `struct Circulo` y una función

Figura 2: Muestra los resultados del LLM para la aplicación de paint.

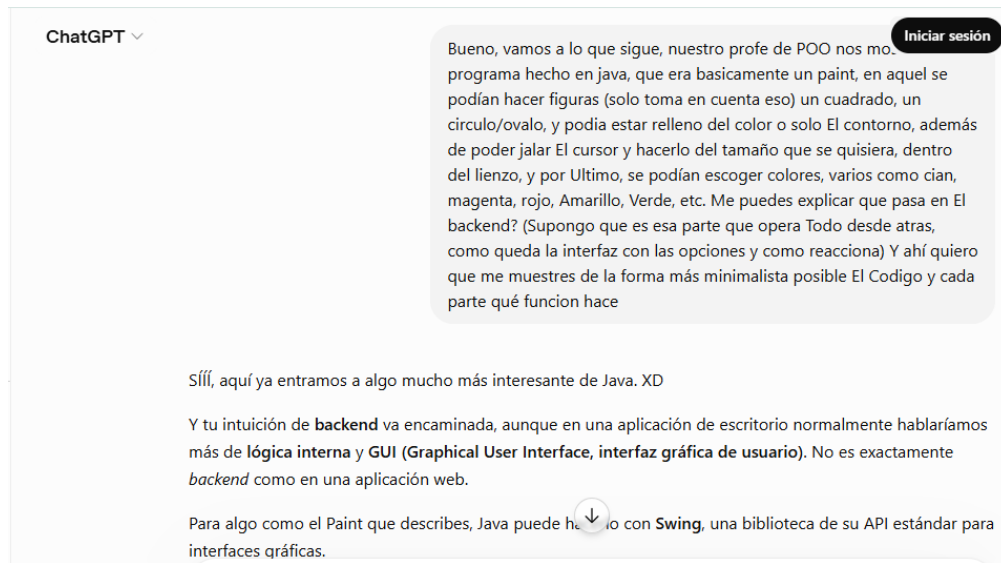


Figura 3: Muestra otra respuesta de la LLM para la aplicación de paint

Figura 4: En la imagen del png muestra todo el modelo gráfico de todas operaciones, de forma visual y que regresa al usuario.

Este es el link para el video de la Práctica 2: [Ver video en YouTube](#)

5. Conclusiones

El análisis de las respuestas de la LLM demuestra que mientras más específica y detallada fuera la salida deseada daba posibles soluciones más acertadas. Además de dar soluciones con la intención de enfocarse en el paradigma de programación orientada a objetos por el contexto del prompt. Se comprobó la importancia del conocimiento conceptual de las estructuras de control y el funcionamiento de la clase Math, ya incluida en java, para realizar operaciones aritméticas en java.

6. Referencias

- [1] Oracle, “Math (Java SE 17 y JDK 17),” Oracle Help Center. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html>. [Consultado: 28-ago-2026].
- [2] Oracle, “Branching Statements — The Java™ Tutorials,” Oracle Technology Network. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/branch.html>. [Consultado: 28-ago-2026].
- [3] Oracle, “Control Flow Statements — The Java™ Tutorials,” Oracle Technology Network. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>. [Consultado: 28-ago-2026].
- [4] Oracle, “Scanner (Java SE 17 y JDK 17),” Oracle Help Center. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Scanner.html>. [Consultado: 28-ago-2026].
- [5] Oracle, “Abstract Methods and Classes — The Java™ Tutorials,” Oracle Technology Network. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html>. [Consultado: 28-ago-2026].
- [6] Oracle, “ArrayList (Java SE 17 y JDK 17),” Oracle Help Center. [En línea]. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html>. [Consultado: 28-ago-2026].
- [7] Oracle, “How to Write a Mouse Listener — The Java™ Tutorials,” Oracle Technology Network. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/uiswing/events/mouselistener.html>. [Consultado: 28-ago-2026].
- [8] Oracle, “Performing Custom Painting — The Java™ Tutorials,” Oracle Technology Network. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/uiswing/painting/>. [Consultado: 28-ago-2026].
- [9] Oracle, “Using Top-Level Containers — The Java™ Tutorials,” Oracle Technology Network. [En línea]. Disponible en: <https://docs.oracle.com/javase/tutorial/uiswing/components/toplevel.html>. [Consultado: 28-ago-2026].